

Personal Finance Tracking System

Comprehensive Software Requirements Specification (SRS) & Technical Design Plan

Document Type:	Engineering Specification & Architecture Design
Target System:	Responsive Web Application
Date:	June 2026
Status:	Baseline Draft

1. Software Requirements Specification (SRS)

1.1 Purpose & Scope

This document specifies the software requirements for the Personal Finance Tracking System website. It details the functional, non-functional, and behavioral requirements of the system to provide a clear operational blueprint for developers, testers, and stakeholders.

The Personal Finance Tracking System is a web-based platform designed to help individuals monitor, manage, and optimize their financial workflows. The system enables users to securely log and track multi-currency transactions, classify income streams, manage expenses, establish dynamic category-based budgets, set specific long-term savings goals, and visualize cash flows through interactive reporting tools.

1.2 User Classes & Characteristics

- **End User:** Individuals seeking to organize their personal finances. They require an intuitive, low-friction responsive interface requiring no prior accounting knowledge.
- **System Administrator:** Privileged internal operators responsible for system health monitoring, infrastructure scaling, and managing global fallback configuration parameters.

1.3 Core Functional Requirements (FR)

1.3.1 User Account & Authentication

- **FR-1.1 (Registration):** The system shall allow new users to register an account using a unique email address, a alphanumeric username, and a secure password.
- **FR-1.2 (Login):** The system shall authenticate users via a secure login portal before granting visibility or structural write-access to any financial database state.
- **FR-1.3 (Password Recovery):** The system shall provide a secure "Forgot Password" self-service pipeline that transmits an asymmetric single-use cryptographic token via email verification.
- **FR-1.4 (Profile Management):** The system shall allow users to update their profile configurations, refresh access credentials, or permanently execute a system account deletion.

1.3.2 Transaction Management (Income & Expenses)

- **FR-2.1 (Add Transaction):** The system shall allow users to log income events (e.g., salary, freelance invoices, asset yields) and expense events (e.g., housing, food, transportation).
- **FR-2.2 (Transaction Ledger Metadata):** For every transaction, the database must accurately store: explicit decimal Amount, Date of occurrence, Category linkage, Payment Method designation, and an optional alphanumeric narrative description.

- **FR-2.3 (CRUD Matrix):** The system shall provide comprehensive user access to read historical ledger grids, update transaction metadata points, or permanently purge individual entries.
- **FR-2.4 (Bulk Actions):** The system shall enable users to perform multi-row selections to delete or recategorize multiple transaction entries simultaneously.

1.3.3 Category Customization

- **FR-3.1 (System Defaults):** The system shall populate all new profiles with a standard set of normalized default categories (e.g., Housing, Food, Utilities, Travel, Entertainment).
- **FR-3.2 (Custom Dimensions):** The system shall grant users the capability to generate unique custom categories assigned with a specific tracking type ('INCOME' or 'EXPENSE').

1.3.4 Budgeting & Financial Goal Tracking

- **FR-4.1 (Monthly Budgets):** The system shall enable users to initialize explicit monetary thresholds on specific expense categories restricted to a calendar month.
- **FR-4.2 (Threshold Alerts):** The system shall continuously compute transaction aggregations against budget limits and trigger visual indicators on the dashboard interface when a category reaches **80%** and **100%** of its threshold limit.
- **FR-4.3 (Savings Milestones):** The system shall allow users to establish distinct milestone targets defining a goal name, a destination dollar value, and an intended completion deadline date.

1.3.5 Analytics & Reporting Visuals

- **FR-5.1 (Real-Time Aggregate Matrix):** The core landing dashboard shall prominently compile and render the user's running Net Balance, total current month income totals, and running monthly expenditure sums.
- **FR-5.2 (Data Filtering Engine):** The system shall allow users to instantaneously filter their visual telemetry views across defined date vectors (weekly, monthly, annual, or custom date-range spans).
- **FR-5.3 (Data Portability):** The system shall render file download bridges enabling users to export structured financial ledgers directly into Comma-Separated Values (CSV), Excel (.xlsx), or print-optimized PDF formats.

1.4 Non-Functional Requirements (NFR)

- **NFR-1.1 (Security Foundations):** All structural plaintext passwords must undergo salted cryptographic hashing via a strong password-hashing function (e.g., bcrypt with a work factor of 10) prior to disk serialization.
- **NFR-1.2 (Transport Layer Encryption):** All data in transit across client endpoints and server API routing interfaces must be wrapped entirely within TLS/HTTPS encryption protocols.
- **NFR-1.3 (Performance Targets):** Under baseline operational parameters, core landing view panels and primary analytics components shall render completely within **2 ext{ seconds}** of initial request invocation.

2. Technical Design Plan

2.1 Tiered System Architecture

The application is structurally isolated into a clean three-tier architecture model, partitioning UI presentation layout layers, deterministic business execution API environments, and standard relational object-relational database targets.

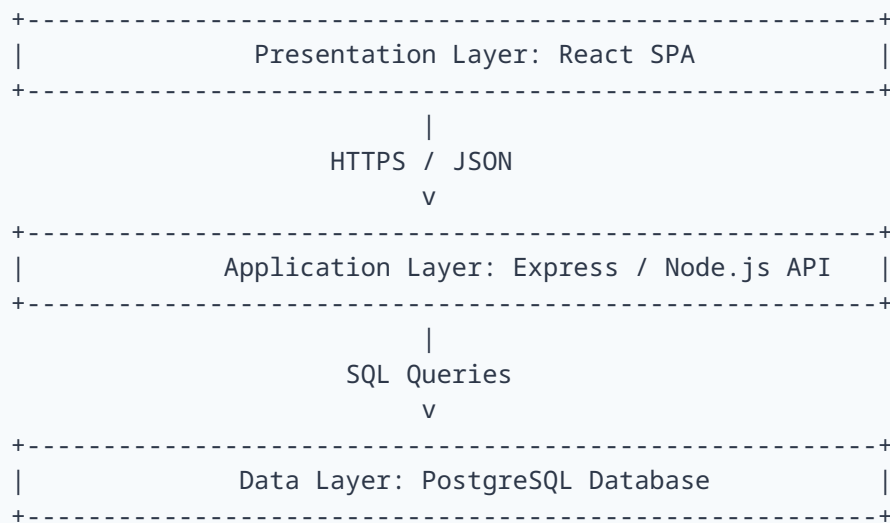


Figure 1: Architectural System Component Matrix

2.2 Database Layer & Object Schemas

To preserve uncompromising transaction isolation and full ACID guarantees, the core storage cluster relies on a structured relational relational framework (PostgreSQL / MySQL).

2.2.1 Data Dictionary Specifications

Table: users

COLUMN NAME	DATA TYPE	CONSTRAINTS	FUNCTIONAL NARRATIVE
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Immutable global unique system user reference identifier.
username	VARCHAR(50)	NOT NULL, UNIQUE	User chosen identity handle rendered across dashboard headers.
email	VARCHAR(100)	NOT NULL, UNIQUE	Validated primary email target routing point for login and recovery.
password_hash	VARCHAR(255)	NOT NULL	One-way salted cryptographic secure string generated via bcrypt.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	System timestamp tracking date-time of account onboarding completion.

Table: categories

COLUMN NAME	DATA TYPE	CONSTRAINTS	FUNCTIONAL NARRATIVE
id	INT	PRIMARY KEY, SERIAL	Auto-incrementing system tracking record identifier key.
user_id	UUID	FOREIGN KEY -> users(id), NULLABLE	Identifies owner profile; NULL tracks core generic system global options.
name	VARCHAR(50)	NOT NULL	Text designation value for groupings (e.g., "Groceries", "Bonus").
type	VARCHAR(10)	NOT NULL	Direction state check constraint: Must read 'INCOME' or 'EXPENSE'.

Table: transactions

COLUMN NAME	DATA TYPE	CONSTRAINTS	FUNCTIONAL NARRATIVE
id	UUID	PRIMARY KEY	Unique ledger entity row reference key.
user_id	UUID	FOREIGN KEY -> users(id), NOT NULL	Target account reference linking transaction row instance context.
category_id	INT	FOREIGN KEY -> categories(id), NOT NULL	Assigned category linkage reference tag.
amount	NUMERIC(12,2)	NOT NULL	High-precision exact currency value representation scalar tracking.
date	DATE	NOT NULL	Effective calendar day index marking transaction event execution.
payment_method	VARCHAR(30)	NOT NULL	Categorization token tracking structure (e.g., 'Credit Card', 'Cash').
description	TEXT	NULLABLE	Free-form raw notes field capturing custom transaction insights.

Table: budgets

COLUMN NAME	DATA TYPE	CONSTRAINTS	FUNCTIONAL NARRATIVE
id	INT	PRIMARY KEY, SERIAL	Auto-indexing identifier key map tracker for budget constraints.
user_id	UUID	FOREIGN KEY -> users(id), NOT NULL	Target tracking context mapped constraint user identifier.
category_id	INT	FOREIGN KEY -> categories(id), NOT NULL	Target expense grouping type assigned to the budget cap rule.
amount_limit	NUMERIC(12,2)	NOT NULL	Maximum money limit assigned for a tracking window context.
month_year	DATE	NOT NULL	Truncated operational month-year coordinate tracking key (e.g., '2026-06-01').

2.3 RESTful Endpoint Ecosystem

All core backend services present declarative HTTP web controller routes parsing application/json payloads. Protected routes mandate presence of a parsing signature token verified within standard request middleware layers.

- POST /api/auth/register — Receives new user credentials, hashes parameters, inserts into base user records, returns success status.
- POST /api/auth/login — Validates signature keys against storage indices and issues signed stateless JSON Web Tokens (JWT).
- GET /api/transactions — Queries ledger rows for active profile tracking parameters. Accommodates parsing filter criteria via URL string attributes: ?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD&category_id=X.
- POST /api/transactions — Appends an item record row to ledger schemas. Triggers background evaluations parsing category budget thresholds.
- DELETE /api/transactions/:id — Dispatches internal target removal routines verifying row ownership before deleting tracking records.
- GET /api/budgets — Compiles set maximum limits alongside calculated execution fields monitoring spent balances across tracking intervals.

2.4 Algorithmic Matrix & Business Calculations

2.4.1 Real-Time Dashboard Net Balance Calculation

To dynamically assemble core metrics displayed on the primary user dashboard panel interface, the system aggregates matching transaction profiles via a deterministic algebraic summation query process:

$$\text{Net Balance} = \sum (\text{Amount}_{\{\text{Income_Transactions}\}}) - \sum (\text{Amount}_{\{\text{Expense_Transactions}\}})$$

2.4.2 Budget Evaluation Workflow Trigger

Upon successful execution of a new expense item entry through the backend route portal, the system triggers a real-time operational evaluation workflow pipeline:

- The system extracts the maximum defined ceiling configuration parameter (**amount_limit**) from matching active tracking configurations.
- The context engine summarizes total expenditures logged under identical classifications during the current processing tracking framework cycle interval (**E_total**).
- The evaluation condition processes the tracking check step rule:

*ext{If } E_{total} \ge 0.80 imes amount_limit \longrightarrow ext{Append Alert Warning Flag to
Response Payload}*